certora

Security Assessment & Formal Verification

Final Report

SOLANA FOUNDATION

P-ATA

October 2026

Prepared for Solana Foundation

certora

Table of Contents

Project Summary **4**
Project Scope 4
Project Overview 4
Protocol Overview 4
Findings Summary 6
Severity Matrix 6
Detailed Findings **7**
Informational Issues 8
I-01. RecoverNested fails on an extra eighth account that ATA ignores 8
I-02. RecoverNested rejects a signing wallet that is 355 bytes and token-owned 8
I-03. Create leaves the new ATA at zero lamports when the rent-exempt minimum is zero 9
I-04. Error code divergences 11
I-05. P-ATA allows 0 as bump seed 12
I-06. Local account-size calculation in P-ATA can drift from the Token-2022 program 13
I-07. Closed owner mint can block nested token recovery 14
I-08. Unbounded account-length hint can transfer excess rent to the ATA owner 14
I-09. Zero-value transfer prevents closing empty non-transferable nested ATAs 15
Formal Verification **16**
Verification Methodology 16
Formal Verification Plan 16
Panicking Behaviour and Panic-Freeness 17
Success Behaviour 17
Error Behaviour 18
CreateWithArgs Behaviour 19
Modelling CPIs 19
Execution Model and Scope of Verification 21
Verification Notations 22
General Assumptions and Simplifications 22
Formal Verification Properties Overview 23
Detailed Properties 26
P-01. Panic Freeness 26
P-02. Panic Subsumption 27
P-03. Success Subsumption 27
P-04. Success Equivalence Lamports 28
P-05. Success Equivalence Accounts 29
P-06. Error Subsumption 30

P-07. Error Code Equivalence....31
P-08. CreateWithArgs Equivalence....32
P-09. CPI Mock Builder and Dispatch Equivalence....33
P-10. CPI Target Mock Faithfulness....35
Disclaimer....37
About Certora....37

certora

Project Summary

Project Scope

Project Name	Repository (link)	First Commit Hash	Latest Commit Hash	Platform
SPL P-ATA	solana-program/associated-token-account	2dc55ee	06b93de	Solana

Project Overview

This document describes the specification and verification of **P-ATA** using the Certora Prover. The work was undertaken from **09/01/2026** to **10/13/2026**.

The following contract list is included in our scope:

pinocchio/interface/src/*.rs
pinocchio/program/src/*.rs
program/src/*.rs

P-ATA is meant to supersede the current ATA program at the same program address. For that to be safe, every client and every account that works with the current program must keep working the same way with P-ATA. The goal of this FV project is to establish that the two programs behave equivalently: on every input, both succeed and leave the same account state, or both fail with the same error. The current program is the reference. Any input on which the two programs disagree are documented as findings on the below pages.

Protocol Overview

On Solana, a wallet does not hold tokens directly. Each token balance lives in a separate token account that the SPL Token program or the Token-2022 program manages. A wallet that holds ten different tokens therefore needs ten token accounts. Without a convention, any party that wants to send tokens to a wallet must first ask the owner which token account to use.

The **Associated Token Account** (ATA) program removes this problem. For each pair of wallet and token mint, the program defines one canonical token account address. The address is a *program derived address* (PDA), which means the program computes it from the wallet address, the token program address, and the mint address. Anyone can compute the address without a lookup. A sender can therefore create the recipient's token account and fund it in one transaction, and the recipient does not need to take part.

The program has three instructions.

- `Create` creates the associated token account for a wallet and a mint. A funding account pays the rent. The instruction fails if the account already exists.
- `CreateIdempotent` does the same, but it succeeds without a change if the account already exists with the expected owner. It fails if the account exists with a different owner.
- `RecoverNested` repairs a mistake. A nested token account is an associated token account whose owner is itself an associated token account instead of a wallet. Tokens sent there are stuck, because no private key can sign for a PDA. The instruction moves all tokens from the nested account to the wallet's correct associated token account, closes the nested account, and returns its lamports to the wallet. Only the wallet that owns the outer account can call it.

P-ATA is a new implementation of the same program that is written with the *Pinocchio* framework instead of the standard solana-program library. The goal is lower compute cost per instruction. P-ATA is intended to replace the current ATA program at the same program address.

P-ATA keeps the same instruction encoding, the same account layout, and the same address derivation as the current program. A client that works with the current program must work with P-ATA without changes. The three instructions above behave the same way.

P-ATA adds one instruction, `CreateWithArgs`, that the current program does not have. It creates an associated token account in either the normal or the idempotent mode. The caller can pass optional hints that let the program skip work: the PDA bump seed, the required account data length, and the rent sysvar as an extra account. If a hint is absent, P-ATA computes the value itself, as the current program does. If a hint is wrong, the instruction must fail rather than create a malformed account.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	9	7	1
Total	9	7	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

The scope of this project was the equivalence of P-ATA and ATA, so most findings (I-01 to I-06) below describe inputs on which the two programs behave differently. In addition, findings I-07 to I-09 concern behaviour that both programs share.

ID	Title	Severity	Status
I-01	RecoverNested fails on an extra eighth account that ATA ignores	Info	Acknowledged
I-02	RecoverNested rejects a signing wallet that is 355 bytes and token-owned	Info	Dismissed
I-03	Create leaves the new ATA at zero lamports when the rent-exempt minimum is zero	Info	Dismissed
I-04	Error code divergences	Info	Acknowledged
I-05	P-ATA allows 0 as bump seed	Info	Fixed
I-06	Local account-size calculation in P-ATA can drift from the Token-2022 program	Info	Acknowledged
I-07	Closed owner mint can block nested-token recovery	Info	Acknowledged
I-08	Unbounded account-length hint can transfer excess rent to the ATA owner	Info	Acknowledged
I-09	Zero-value transfer prevents closing empty non-transferable nested ATAs	Info	Acknowledged

Informational Issues

I-01. RecoverNested fails on an extra eighth account that ATA ignores

Description: The two programs read a different number of accounts for `RecoverNested`. ATA reads seven and never looks at what follows. Its docs list seven and its instruction builder sends seven. An eighth account has no effect on the result.
P-ATA reads an eighth account and uses it as a second token program. That account is used in four places, and in each one it stands where ATA uses account 6: two are address derivations for the nested ATA and for the destination ATA, and two are ownership checks for the nested ATA and for the nested mint.
A divergence where P-ATA fails but ATA succeeds is reachable when the eighth account differs from account 6, and the nested ATA and mint still belong to account 6. That is an ordinary single token program recovery with one stray account appended. All four checks then run against a token program that ATA never used. The nested derivation no longer matches the supplied ATA. Three diverging outcomes are reachable:

- The call fails with `InvalidSeeds` (`derived_nested_ata != *nested_ata.address()` or `derived_destination_ata != *destination_ata.address()`).
- If the addresses match in above checks, it can still fail with an `IllegalOwner` error (`!nested_ata.owned_by(nested_token_program.address())`).
- Finally, if the derivation finds no viable bump, P-ATA panics since `derive_address` is called with the new `nested_token_program` from the eight account, which in turn calls `derive_address_and_bump_seed`, which unwraps that case with `expect`.

Customer's response: Acknowledged and assessed as benign. The instruction is rarely used, and no functionality depends on trailing accounts being ignored.

I-02. RecoverNested rejects a signing wallet that is 355 bytes and token-owned

Description: `RecoverNested` takes the wallet that owns the owner ATA as the fifth account, and that account has to authorize the call. The two programs decide whether it did in different ways. ATA simply checks whether the wallet is the signer:

```rust
if !wallet_account_info.is_signer {
       msg!("Wallet of the owner associated token account must sign");
       return Err(ProgramError::MissingRequiredSignature);
}
```

In particular, it does not support multisig wallets.
P-ATA supports multisig and decides between the two ways of authorizing based on the wallets size and owner:

```rust
if wallet.data_len() == Multisig::LEN && (wallet.owned_by(&pinocchio_token::ID) ||
wallet.owned_by(&pinocchio_token_2022::ID)) {
       let wallet_signers = remaining.get(1..).unwrap_or_default();
       validate_multisig_wallet(wallet, wallet_signers)?;
} else if !wallet.is_signer() {
       return Err(wallet_missing_required_signature());
}
```

Any wallet account of exactly `Multisig::LEN` size owned by SPL Token or Token-2022 takes the multisig path, and because the second arm is `else if`, its own signature is never looked at. For such wallets, P-ATA can reject the call to `RecoverNested` during `validate_multisig_wallet` if either the wallet is not initialized as a multisig, or not enough signers were passed as trailing accounts. Thus, if the wallet itself is the signer of the transaction, then the call passes on ATA whereas P-ATA rejects it.

Customer's response: Dismissed with the reason that multisig accounts are, by definition, identified by ownership and their reserved byte length.

I-03. Create leaves the new ATA at zero lamports when the rent-exempt minimum is zero

Description: The two programs fund a new associated token account by different amounts when the rent-exempt minimum is zero.
ATA raises the funding target to at least one Lamport. If the account already holds lamports, it tops it up to that floor:

```rust
let required_lamports = rent
```

```
        .minimum_balance(space)
        .max(1)
        .saturating_sub(new_pda_account.lamports());
```

Otherwise, if it holds none, the same floored amount goes directly in the `CreateAccount` CPI:

```
Rust
&system_instruction::create_account(
        payer.key,
        new_pda_account.key,
        rent.minimum_balance(space).max(1),
        space as u64,
        owner
)
```

P-ATA instead hands the task to Pinocchios' `CreateAccountAllowPrefund::with_minimum_balance` CPI builder prior to executing the account creation CPI.

```
Rust
CreateAccountAllowPrefund::with_minimum_balance(
        payer,
        associated_token_account,
        account_len,
        token_program.address(),
        rent_sysvar
)?.invoke_signed(&[signer])?;
```

Within that builder, the required Lamports are computed as

```
Rust
// here `to` is `associated_token_account`
let lamports = required_lamports.saturating_sub(to.lamports());
```

so no floor applies. A rent-exempt minimum of zero and an ATA at zero lamports produce a system instruction that allocates and assigns the account and transfers nothing into it. The runtime then deletes it at the end of the transaction because its balance is zero. This is the case exactly when the rent rate is zero.

Customer's response: Dismissed with the rationale that zero-rent environments do not need to be considered.

I-04. Error code divergences

Description: The two programs fail on some inputs with different error codes. On each input below, both programs reject the transaction, but with a different error code. A failing CPI aborts the transaction with the error of the invoked program. To the user, that abort looks the same as a returned error.

- **Divergent account initialization sequences** (`Create`, `CreateIdempotent`). Associated token account creation is different in ATA and P-ATA. In ATA the sequence is `transfer`, `allocate`, then `assign`, whereas in P-ATA calls `CreateAccountAllowPrefund`, which the system program executes as `allocate`, `assign`, then `transfer`. If `transfer` and `allocate` or `assign` are both returning an error for the inputs passed to `process_create_associated_token_account`, then ATA would return the `transfer` error, while P-ATA will return the `allocate` or `assign` error. The only way this can happen is when it is part of a transaction that grows account sizes beyond `MAX_ACCOUNT_DATA_GROWTH_PER_TRANSACTION`. This cannot happen in a transaction only using calls to (P-)ATA.
- **Account size check divergence** (`Create`, `CreateIdempotent`). When passing as input a Token-2022-owned, 82-byte, uninitialized mint and the payer has insufficient funds to create the ATA then ATA returns an `InvalidMint` error from the CPI. P-ATA instead of the CPI does a local size calculation and later fails with an insufficient-funds error when trying to initialize the account.
- **Unknown token program** (`Create`, `CreateIdempotent`). If the token program account is neither SPL Token nor Token-2022, and no ATA exists yet, then P-ATA returns `IncorrectProgramId` before any CPI is executed. ATA does not check the token program. It proceeds to call `GetAccountDataSize` on the program and then `create_account`, so the transaction fails with whatever error comes first: the error of that program, or a system program or runtime error from `create_account`.
- **Unknown token program** (`RecoverNested`). If the owner token program account is neither SPL Token nor Token-2022, then the checks in P-ATA only compare account owners against that account, so they pass. P-ATA then returns `IncorrectProgramId` from the program check in `invoke_signed_with_program`, before any CPI is executed. ATA does not check the token program. It unpacks the owner ATA, the nested ATA and the nested mint. The transaction can fail on the first unpack error, for example `UninitializedAccount`. If all three accounts unpack, it fails with the error of that program from the transfer CPI.

- **Uninitialized nested mint** (`RecoverNested`). If the nested mint is a Token-2022 mint that is not initialized, and the destination ATA does not exist yet, then ATA unpacks the nested mint before any CPI and returns `UninitializedAccount`. P-ATA does not check that the mint is initialized, so it calls `TransferChecked`. Token-2022 checks the owner of the destination before it unpacks the mint. The destination is still owned by the system program, so the transaction fails with IncorrectProgramId.
  Note: This state is reachable from a Token-2022 mint with a close authority and zero supply. The close authority closes the mint, and the holder of the mint keypair creates a new 82-byte Token-2022 account at that address without initializing it. The nested ATA for the old mint still exists.

Customer's response: Acknowledged and assessed as benign.

I-05. P-ATA allows 0 as bump seed

Description: ATA and P-ATA search different bump-seed ranges when they derive the associated token account address. ATA derives the address through the `sol_try_find_program_address` syscall. The runtime implementation of that syscall starts at bump 255 and makes 255 attempts, so it tests bumps 255 down to 1 and never tests bump 0. P-ATA derives the address through `Address::derive_program_address` from the `solana-address` crate, which runs the loop below and tests bump 0 after bumps 255 to 1 fail:

```rust
let mut bump = u8::MAX;
loop {
	let address = Self::derive_address(seeds, Some(bump), program_id);
	if !address.is_on_curve() {
		return Some((address, bump));
	}
	if bump == 0 {
		return None;
	}
	bump -= 1;
}
```

If a wallet, token program and mint produce an on-curve address for every bump from 255 to 1 and an off-curve address for bump 0, ATA finds no PDA and aborts, while P-ATA derives the

bump-0 address and creates the account. The two programs then disagree on whether the input is accepted. No seed combination with this property is known, and finding one is astronomically unlikely, because it requires 255 consecutive on-curve candidates.

Recommendation: Make P-ATA return None before it derives with bump 0, so that it follows the same bump range as ATA.

Customer's response: Acknowledged and fixed.

Fix Review: Fixed by patching `Address::derive_program_address` to stop at bump 1 (commit 118c2eb0 in solana-sdk). Additionally, the P-ATA dependency to solana-address was bumped to version 2.9.0 to include that change (commit 5a3942de).

I-06. Local account-size calculation in P-ATA can drift from the Token-2022 program

Description: The two programs obtain the data size of a new Token-2022 associated token account differently. ATA always calls GetAccountDataSize on the token program:

```rust
invoke(
    &spl_token_2022_interface::instruction::get_account_data_size(
        spl_token_program.key,
        mint.key,
        extension_types,
    )?,
    &[mint.clone(), spl_token_program.clone()],
)?;
```

P-ATA computes the size locally from the mint data and only falls back to the CPI when the local calculation fails:

```rust
if let Ok(len) = mint.try_borrow().and_then(|mint_data| {
    try_calculate_account_len_from_mint_data(&mint_data, &[SplExtensionType::ImmutableOwner])
}) {
    return Ok(len as u64);
}
get_account_data_size_cpi(mint, token_program)
```

Both the on-chain Token-2022 program and P-ATA rely on the same functions `try_calculate_account_len_from_mint_data` from the spl-token-2022-interface crate, so the two sizes agree as long as the two copies of that function behave the same. Token-2022 can be upgraded independently of P-ATA. If an upgrade changes the size that an existing mint extension requires, the local calculation in P-ATA still succeeds and returns the old size, while ATA receives the new size from the CPI. If the old size is too small, P-ATA fails during initialization where ATA succeeds. If it is large enough, both succeed but allocate different sizes and charge different rent.

Customer's response: Acknowledged.

I-07. Closed owner mint can block nested token recovery

Description: Both ATA and P-ATA require `owner_token_mint` to still be owned by the selected token program in `process_recover_nested`. A Token-2022 mint with `MintCloseAuthority` may be explicitly closed by its close authority once its supply is zero, after which it is owned by the System Program. A zero supply alone does not automatically close the mint.

Closing the owner mint does not close its existing owner ATA. That ATA may still control a nested ATA containing an arbitrary balance of an unrelated mint. In this state, the ownership check rejects recovery even though the closed owner mint is only used as a PDA seed. If the mint cannot be recreated at the same address and no alternative authority can recover the assets, both the nested tokens and the nested ATA's rent become inaccessible through either implementation.

Customer's response: Acknowledged; the issue may be fixed in a later rewrite of the recovery function.

I-08. Unbounded account-length hint can transfer excess rent to the ATA owner

Description: For Token-2022, `CreateWithArgs` uses the caller-provided `account_len` directly instead of deriving the required size from the mint. An oversized hint can allocate up to the System Program's 10 MiB limit and charge the funder roughly 73 SOL in rent. Because the resulting token account belongs to the wallet, the wallet can close it and receive those lamports.

The funder must authorize the operation as a signer, so an attacker cannot charge an arbitrary account. Exploitation requires a sponsored transaction or CPI integration that signs for a third-party funder while allowing an untrusted user to control the wallet and `account_len` hint.

Recommendation: When the funder and wallet differ, ignore the hint and derive the required account size from the mint. Alternatively, validate the hint against the derived size or enforce a conservative maximum allocation.

Customer's response: Acknowledged but won't be fixed.

I-09. Zero-value transfer prevents closing empty non-transferable nested ATAs

Description: Both ATA and P-ATA always execute `TransferChecked` before `CloseAccount`, even when the nested ATA has a zero balance. Token-2022 rejects transfers from accounts created for a `NonTransferable` mint regardless of the transfer amount, so the zero-value transfer fails and the close operation is never executed.

An empty non-transferable account could otherwise be closed successfully. However, the nested ATA is controlled by the off-curve `owner_ata` PDA, so the wallet cannot sign a separate close instruction. Its rent therefore remains inaccessible through either implementation. The same problem can affect other account or mint states that reject zero-value transfers but permit closing.

Recommendation: When the nested ATA's balance is zero, skip `TransferChecked` and execute only `CloseAccount`.

Customer's response: Acknowledged; the issue may be fixed in a later rewrite of the recovery function.





Formal Verification

Verification Methodology

We performed verification of the protocol using the Certora Prover which is based on Satisfiability Modulo Theories (SMT).
In short, the Certora Prover works by compiling formal specifications written in the Certora Verification Language (CVLR) and Kamino's implementation source code written in Rust.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVLR holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors/initialization functionality as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective initialization function. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant. Each such case results in one rule.

Formal Verification Plan

The objective of this project is to establish that P-ATA can stand in for ATA without breaking any of its guarantees. This means: on every input, P-ATA ends the same way that ATA ends, with the same error where both reject, and with the same account state where both succeed. The plan below states the properties that need to hold per outcome and per direction.

Note that this section is a general overview over the verification plan. Additional restrictions and assumptions apply to individual rules, which are not discussed here. Please refer to General Assumptions and Simplifications and the assumptions stated for individual rules in Detailed Properties.

Panicking Behaviour and Panic-Freeness

All entry points (both ATA and P-ATA) are panic-free except for the following known exceptions:

- The computation of PDAs may panic. ATA calls `Pubkey::find_program_address` and Pinocchio calls `AssociatedTokenPda::derive_address_and_bump_seed`. Both panic if neither of the 256 possible and sequential checked bump seeds yields a correct PDA for the given seeds.
- ATA`s `Create` and `CreateIdempotent` panic in `Rent::minimum_balance` if the rent sysvar holds a `lamports_per_byte` large enough to overflow the rent computation or if the token program reports an account size above the 10 MiB account limit.

We therefore verify that

(Property P-01) Apart from the known exceptions, both programs are panic-free.

Additionally, we verify that even when above known exceptions are allowed to occur, P-ATA never panics on any given input accounts that ATA does not panic on:

(Property P-02) P-ATA panics only if ATA panics on the same input accounts.

Note that the converse does not hold, but this case is benign or can be considered an improvement: P-ATA allows strictly more non-panicking inputs.

Success Behaviour

It is expected that any call that succeeds on ATA, then the corresponding call on P-ATA must also succeed. We verify this in

(Property P-03) P-ATA returns `Ok(())` whenever ATA returns success on the same entry point with identical inputs.

If both calls do succeed, then additionally we verify that the effect to the passed accounts are equivalent in the following properties

(Property P-04) If both programs succeed on identical input accounts, then all passed accounts must be left with the same Lamport balance.

(Property P-05) If both programs succeed on identical input accounts, then the passed token account(s) that the respective entry points write to must end in an identical state.

Error Behaviour

Analogously to the success path, P-ATA is expected to error only if ATA errors on identical inputs on the same entry point. Additionally, we demand that if both programs do return an error, then for the same reason, i.e., they return the same error variant/code. We capture both in the following two properties:

(Property P-06) P-ATA returns `Err(...)` only if ATA returns `Err(...)` on the same entry point with identical inputs.

(Property P-07) If P-ATA returns `Err(e1)` and ATA returns `Err(e2)` on the same entry point with identical inputs, then `e1 == e2`.

Both above properties treat aborted transactions from failing CPIs as `Err(..)` with a corresponding custom error code instead. In other words, under verification, if a CPI callee returns an `Err(...)`, which would usually lead to the transaction being aborted, we propagate the error back to the caller and resume ordinarily. This means that if a CPI target returns `Err(e)`, then P-ATA or ATA function making the CPI also returns `Err(e)` as a result. Section Modelling CPIs below details on how this is achieved.

CreateWithArgs Behaviour

P-ATA introduces a new entry point `CreateWithArgs`, that works like `Create/CreaeIdempotent`, but allows the caller to pass optional hints for values that otherwise need to be derived. We establish that this function can be used interchangeably with `Create` or `CreateIdempotent` whenever the passed hints are either `None` or coincide with what `Create` and `CreateIdempotent` derive themselves:

(Property P-O8a) `CreateWithArgs` behaves equivalent to `Create` when the mode is `Always`, the bump hint is `None` or the bump that `Create` derives, and account length hint is `None` or the size that `Create` derives.
(Property P-O8b) `CreateWithArgs` behaves equivalent to `CreateIdempotent` when the mode is `Idempotent`, the bump hint is `None` or the bump that `CreateIdempotent` derives, and account length hint is `None` or the size that `CreateIdempotent` derives.

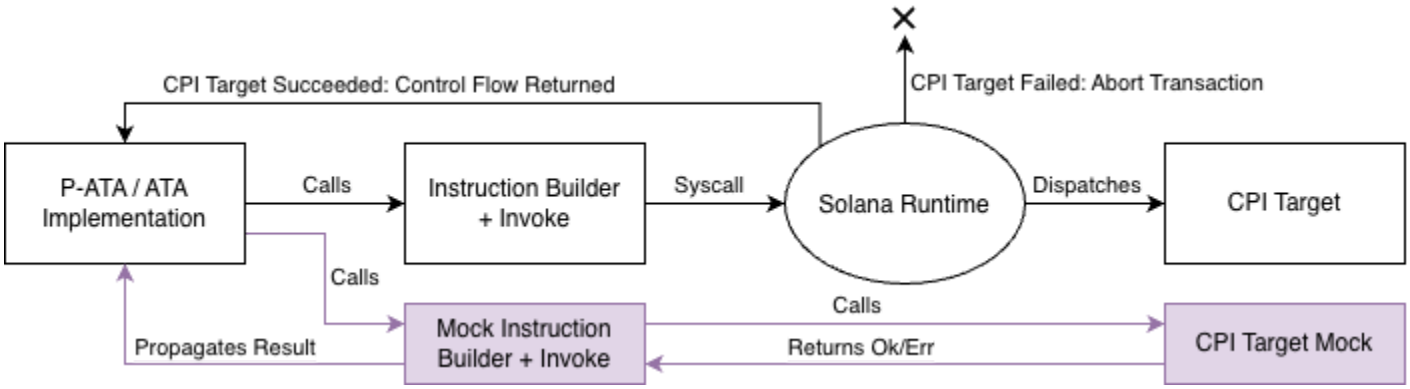
Note that `CreateWithArgs` additionally the the rent sysvar as a third optional parameters. Since the rent instance is obtained through a syscall rather than through a computation, we simply assume in both above rules that this is always the same rent instance that `Create(Idempotent)` uses.

Modelling CPIs

Both ATA and P-ATA rely on cross-program invocations (CPIs) of the System program and the token programs (SPL and Token-2022). On chain, a CPI ends in the `sol_invoke_signed_c` syscall. The runtime dispatches the instruction to the callee, the callee is executed, and the runtime either hands control back to the caller or aborts the transaction with the callee's error. Under the prover, which executes code symbolically, the syscall is a black box: the callee never runs, the passed accounts never change, and the result is an unconstrained value. A rule that looks at the state after a CPI would therefore see nothing, and a rule that looks at the result would see arbitrary outcomes.

We close this gap by replacing the CPI path with Rust code that the prover can run. On the P-ATA side, the instruction builders from pinocchio-system and pinocchio-token, such as `CreateAccountAllowPrefund` and `InitializeAccount3`, are swapped for mock builders with the same signatures. On the ATA side, `solana_cpi::invoke` and `invoke_signed` keep their borrow checks, and only the syscall is swapped out. The ATA builders for the System

program instructions and for `initialize_account3` are swapped as well, because their serialization code is not tractable for the prover; the mocks write the same bytes directly. In both cases the mock does not issue a syscall and instead calls a model of the callee instruction directly.
We include one model per System program and token program (SPL Token and Token-2022) instruction that ATA and P-ATA invoke. All mock CPI targets model the on-chain behaviour with the same checks and the same account writes in the same order. Both implementations call the same models, so a difference between the two runs can only come from the program code under verification, not from the model. The idea is visualized in the following diagram:



One behaviour is modelled differently on purpose. On chain, a failing callee aborts the whole transaction and the caller never resumes. In the model, a failing callee returns `Err` with the callee's error code to the caller, and the caller propagates it. This keeps the abort observable as a result value, so the equivalence rules can compare the error code that each implementation would have aborted with. The rules that compare the state after a success assume that every CPI returned Ok, which matches the on-chain behaviour.

The faithfulness of the mocks is verified separately, by comparing the mocked CPI path against the real one up to the CPI itself, and by verifying that the mock targets are functionally equivalent to the CPI targets that would be called on chain:

(Property P-09) The mocked CPI path (instruction builders and invoke_signed) is functionally equivalent to the real one up to the CPI itself.

(Property P-10) The CPI target mocks are functionally equivalent to the processor code of the real CPI targets (System program, SPL Token, Token-2022) up to batching.

The following parts of the CPI path are not verified and assumed correct.

- The runtime's own CPI handling: serialization into the syscall format, the privilege checks before dispatch, and the callee's entrypoint that decodes the instruction data are not covered. The CPI target mocks take typed arguments directly and are compared against the processor functions behind that entrypoint (at pinned versions of the system program, P-Token and Token-2022).
- The batching: P-ATA sends two instruction pairs as one batch instruction, the Token-2022 account initialization in Create and the transfer and close in RecoverNested, and the mock calls the two models in sequence instead, assuming equivalence between sequential and batched calls.

Execution Model and Scope of Verification

For tractability, the entrypoint for verification are the processor functions for both P-ATA and ATA. That is, rules do not run `process_instruction` but they call the processing functions (`process_create_associated_token_account` and `process_recover_nested` on both sides, and the `CreateWithArgs` path on the P-ATA side). The dispatch in `process_instruction`, which decodes the discriminator and forwards to these functions, was audited manually and is not covered by a rule.

Every rule builds its accounts with unconstrained content: address, owner, lamports, data and flags are all symbolic, under a bound on the data length. For the pairwise rules each account is created once and copied, so both implementations run on the same input. The create rules pass the **six** documented accounts, the recover rules pass **seven**, **eight** and **nine** accounts to cover the optional nested token program and a trailing account, and the `CreateWithArgs` rules pass **six** or **seven** accounts to cover the optional rent sysvar. The panic-freeness rules pass any count from zero to ten.

Values that come from the runtime rather than from the accounts are supplied by oracles. An oracle is a single unconstrained value that both implementations read on the same input. This applies to the PDA derivation, the rent-exempt minimum, and the account size Token-2022 reports for a mint. Without the oracles, the prover could answer the same syscall or CPI differently for the two implementations, for example return two different addresses for the same

PDA seeds, and report a deviation that cannot happen on chain. Cross-program invocations are modelled as described in the previous section. Everything else runs as the compiled program code.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counterexample exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

- We use Solana Platform Tools v1.5.3 for verification
- Instruction data is already parsed. All rules call the processing functions directly, as described in Execution Model and Scope of Verification.
- Rules that compare the two implementations pass a fixed number of accounts: six for `Create` and `CreateIdempotent`, and seven, eight or nine for `RecoverNested` (one rule per option). Inputs with fewer or more accounts are out of scope for formal verification and were manually audited.
- Every account is its own memory. An account passed twice is treated as two accounts that do not share lamports or data.
- Unless a property states otherwise, cross-program invocations run the CPI target mocks instead of the real programs.
- The rules treat every informational finding in Detailed Findings as benign and suppress the exact cases that would trigger it, so that these known deviations do not show up as violations.

- The following assumptions and simplifications were necessary to keep verification tractable and globally apply:
    - In the rules that compare an ATA with a P-ATA implementation, every account holds at most 10240 bytes of data. In all other rules the bound is 8 MiB.
    - For `RecoverNested`, the nested mint holds no extension data. Unpacking a mint with extensions scans the padding bytes, which is unfeasible to verify
    - Both implementations take the Token-2022 account size from an account-size oracle, which returns one size per run regardless of the mint. In P-ATA, the function `get_token_2022_account_data_size` is replaced by a mock that returns the oracle value directly, due to a prover incompatibility.

Formal Verification Properties Overview

ID	Title	Property	Status
P-01	Panic Freeness	Apart from the known exceptions, both programs are panic-free.	Verified
P-02	Panic Subsumption	P-ATA panics only if ATA panics on the same input accounts.	Verified
P-03	Success Subsumption	P-ATA returns `Ok(())` whenever ATA returns success on the same entry point with identical inputs.	Verified
P-04	Success Equivalence Lamports	If both programs succeed on identical input accounts, then all passed accounts must be left with the same Lamport balance.	Verified
P-05	Success Equivalence Accounts	If both programs succeed on identical input accounts, then the passed token account(s) that the respective entry	Verified

		points write to must end in an identical state.	
P-06	Error Subsumption	P-ATA returns `Err(...)` only if ATA returns `Err(...)` on the same entry point with identical inputs.	Verified
P-07	Error Code Equivalence	If P-ATA returns `Err(e1)` and ATA returns `Err(e2)` on the same entry point with identical inputs, then `e1 == e2`.	Verified
P-08a	CreateWithArgs Create Equivalence	`CreateWithArgs` behaves equivalent to `Create` when the mode is `Always`, the bump hint is `None` or the bump that `Create` derives, and account length hint is `None` or the size that `Create` derives.	Verified
P-08b	CreateWithArgs CreateIdempotent Equivalence	`CreateWithArgs` behaves equivalent to `CreateIdempotent` when the mode is `Idempotent`, the bump hint is `None` or the bump that `CreateIdempotent` derives, and account length hint is `None` or the size that `CreateIdempotent` derives.	Verified
P-09	CPI Mock Builder and Invoke Equivalence	The mocked CPI path (instruction builders and `invoke_signed`) is functionally equivalent to the real one up to the CPI itself.	Verified

P-10	CPI Mock Target Equivalence	The CPI target mocks are functionally equivalent to the processor code of the real CPI targets (System program, SPL Token, Token-2022) up to batching	Verified

Detailed Properties

P-01. Panic Freeness

Status: Verified	No entry point of either implementation (P-ATA and ATA) panics, apart from the known exceptions listed in Panicking Behaviour and Panic-Freeness. **Assumptions**: • The rules pass 0 to 10 accounts. • CPIs run the real `invoke` code, not the mocks. CPI targets are not executed.

Rule Name	Status	Description	Link to rule report
ata_panic_free_create	Verified	*ATA `Create` is panic-free.*	*Report*
ata_panic_free_create_idempotent	Verified	*ATA `CreateIdempotent` is panic-free.*	
pata_panic_free_create	Verified	*P-ATA `Create` is panic-free.*	
pata_panic_free_create_idempotent	Verified	*P-ATA `CreateIdempotent` is panic-free.*	
pata_panic_free_create_with_args	Verified	*P-ATA `CreateWithArgs` is panic-free.*	
pata_panic_free_process_recover_nested	Verified	*P-ATA `RecoverNested` is panic-free.*	
ata_panic_free_recover_nested	Verified	*ATA `RecoverNested` is panic-free.*	*Report*

P-02. Panic Subsumption

Status: Verified	If a P-ATA entry point panics on some input, the corresponding ATA one panics on the same input. One rule per entry point pair (create, create idempotent, recover nested). CPI targets are not executed. This only covers the known panic-sites listed in Panicking Behaviour and Panic-Freeness.

Rule Name	Status	Description	Link to rule report
pata_panics_only_if_ata_panics_{entry point}	Verified	*P-ATA panics only if ATA panics on {entrypoint}, which is either of* `Create`, `CreateIdempotent`, *or* `RecoverNested`.	*Report*

P-03. Success Subsumption

Status: Verified	On any input where an ATA entrypoint returns `Ok(..)`, the corresponding P-ATA implementation returns an `Ok(..)` on the same inputs. This means, there is no input that the ATA implementation accepts which under the P-ATA implementation leads to an error.

Rule Name	Status	Description	Link to rule report
pata_success_if_ata_success_{entry point}	Verified	*P-ATA succeeds if ATA succeeds on {entrypoint}, which is either of* `Create`, `CreateIdempotent`, *or* `RecoverNested` *with seven, eight, or nine accounts passed.*	*Report*

P-04. Success Equivalence Lamports

Status: Verified	Equivalence between the two ATA and P-ATA on inputs where both corresponding entry points return `Ok(..)`. Verifies that every account holds the same lamports after both runs.

Rule Name	Status	Description	Link to rule report
success_equiv_lamports_{entrypoint}	Verified	*Each pair of corresponding accounts holds the same amount of Lamports if P-ATA and ATA succeeds on {entrypoint}, which is either of `Create`, `CreateIdempotent`, or `RecoverNested` with seven, eight, or nine accounts passed.*	*Report*

P-05. Success Equivalence Accounts

Status: Verified	Equivalence between the two ATA and P-ATA on inputs where both corresponding entry points return `Ok(..)`. Verifies that token accounts the entry point writes are identical after both runs. Note: It is intractable to the raw byte arrays of two associated token accounts for equality. Instead, we assert the data lengths match, parse both accounts, and compare the following fields for equality: • Owner • Mint • Amount • Delegate • Delegate Amount • State • Native Amount • Close Authority • Has Immutable Owner

Rule Name	Status	Description	Link to rule report
success_equiv_ata_{entrypoint}	Verified	*After calling {entrypoint} on the same inputs for ATA and P-ATA, the corresponding associated token account that the entry point writes to are identical in the post state (by above measure).* *The {entrypoint} is either of `Create`, `CreateIdempotent`, or `RecoverNested` with seven, eight, or nine accounts passed.* *`Create` and `CreateIdempotent` compare the associate token account passed at index one, `RecoverNested` those passed at index two and three.*	*Report*

P-06. Error Subsumption

Status: Verified	On any input where a P-ATA entry point returns `Err(..)`, the corresponding ATA implementation returns an `Err(..)` on the same inputs. This means, there is no input that the ATA implementation accepted which under the P-ATA implementation leads to an error. Note that aborts (from a failing CPI) are treated as ordinary errors (see Modelling CPIs), so transaction aborts are subsumed in this rule.

Rule Name	Status	Description	Link to rule report
pata_error_only_if_ata_error_{entry point}	Verified	*P-ATA errors only if ATA errors on {entrypoint}, which is either of `Create`, `CreateIdempotent`, or `RecoverNested` with seven, eight, or nine accounts passed.*	*Report*

P-07. Error Code Equivalence

Status: Verified	Error code equivalence between the two implementations on inputs where both runs fail: both fail the call with the same error code. Codes are compared in! the encoding `u64::from(ProgramError)`. Note that aborts (from a failing CPI) are treated as ordinary errors (see Modelling CPIs), so transaction aborts are subsumed in this rule.

Rule Name	Status	Description	Link to rule report
error_equiv_create	Verified	*When calling `Create` on ATA and P-ATA with the same input accounts, and both return an error, then the error code is the same.*	*Report*
error_equiv_create_idempotent	Verified	*When calling `CreateIdempotent` on ATA and P-ATA with the same input accounts, and both return an error, then the error code is the same.*	*Report*
error_equiv_recover_nested_7	Verified	*When calling `RecoverNested` (with seven passed accounts) on ATA and P-ATA with the same input accounts, and both return an error, then the error code is the same.*	*Report*
error_equiv_recover_nested_8	Verified	*When calling `RecoverNested` (with eight passed accounts) on ATA and P-ATA with the same input accounts, and both return an error, then the error code is the same.*	*Report*
error_equiv_recover_nested_9	Verified	*When calling `RecoverNested` (with nine passed accounts) on ATA and P-ATA with the same input accounts, and both return an error, then the error code is the same.*	*Report*

P-08. CreateWithArgs Equivalence

Status: Verified	`CreateWithArgs` against the plain entry point of its mode: `Create` for `CreateMode::Always`, `CreateIdempotent` for `CreateMode::Idempotent`. The bump hint is `None` or the bump the plain path derives itself, the account length hint is `None` or the size the plain path computes itself, and no rent sysvar account is passed. Two rules per mode: • *result*: both runs return `Ok(..)`, or both return `Err(..)` with the same error code. • *state*: on inputs where both runs return `Ok(..)`, every account holds the same lamports and the associated token accounts are identical (by the same measure as used in Property P-05).

Rule Name	Status	Description	Link to rule report
create_with_args_equiv_result_create	Verified	*Calling `CreateWithArgs` and `Create` on the same inputs, then both runs return `Ok(..)`, or both return `Err(..)` with the same error code.*	*Report*
create_with_args_equiv_state_create	Verified	*Calling `CreateWithArgs` and `Create` on the same inputs, and both return `Ok(())`, every account holds the same lamports and the associated token accounts are identical.*	
create_with_args_equiv_result_create_idempotent	Verified	*Calling `CreateWithArgs` and `CreateIdempotent` on the same inputs, then both runs return `Ok(..)`, or both return `Err(..)` with the same error code.*	
create_with_args_equiv_state_create_idempotent	Verified	*Calling `CreateWithArgs` and `CreateIdempotent` on the same inputs, and both return `Ok(())`, every account holds the same lamports and the associated token accounts are identical.*	

P-09. CPI Mock Builder and Dispatch Equivalence

Status: Verified	Each mocked piece of the CPI path (see Modelling CPIs) is verified against the real code it replaces by a rule that runs both on the same symbolic input. On the P-ATA side these are the instruction builders, which also perform the *invoke*. On the ATA side these are the System program instruction builders and the `invoke_signed` function of solana-cpi, which the processors call with the built instruction. The rules compare the returned result and the state of the passed accounts. The invoked program is disabled in these rules, so the comparison stops where the callee would run.

Rule Name	Status	Description	Link to rule report
mock_ata_{instr}_instruction_matches_real	Verified	*Verifies the mocked `solana_system_interface::instruction` builder {instr} (one of `create_account`, `transfer`, `allocate` and `assign`) returns an Instruction with the same program id, account metas and discriminator as the real builder.*	*Report*
mock_ata_invoke_{instr}_matches_real	Verified	*Verification of the `invoke_signed` mock reached by ATA. Checks the mocked `solana_cpi::invoke_signed` against the real one. Each rule builds one of the instructions {instr} the ATA processors send (the four system instructions `create_account`, `transfer`, `allocate`, `assign`, and the token instruction `initialize_immutable_owner`, `initialize_account3`, `transfer_checked`, `close_account`) and passes it to both the real and the mocked `invoke_signed`.* *Compared are the returned results and the account state. The rules assert that if the real call returns `Ok(())`, the mock returns `Ok(())`, and if the mock returns `Err(e)`, the real call returns the same `Err(e)`. Both calls must leave the owner, lamports and data*	

		length of every passed account unchanged. Neither call may panic.	
mock_pata_{instr}_matches_real	Verified	*Verifies the mocked `pinocchio-system` and `pinocchio-token` builders `CreateAccountAllowPrefund`, `InitializeAccount` and `InitializeAccount3`, and the two `Batch` builders behave like the real builder on copies of the same accounts.* *Compared are the returned results (both must either return Ok(()) or Err(e)) and panic-freeness.*	

P-10. CPI Target Mock Faithfulness

Status: Verified	Rules verifying the CPI mocks against the processor functions of the real 2022 implementation. Each rule runs the model and the processor on copies of the same accounts. Verifies that either both succeed and leave the accounts in the same state, or both fail with the same error. Note: Since the real processor functions are not the in scope of the repository, we use a vendored copy instead, taken verbatim from the at the time stable releases:: • Token-2022: `solana-program/token-2022` ○ Commit / Tag: `9bc02757` / `program@v11.0.0` • SPL: `solana-program/token` ○ Commit / Tag: `a94dd197` / `p-token@v1.0.0` • System: `anza-xyz/agave` ○ Commit / Tag: `c9c6f328` / `v4.2.2` Minimal edits to the vendored copies were necessary to make verification possible. All edits keep soundness and are thoroughly documented in sources..

Rule Name	Status	Description	Link to rule report
token22_{function}_model_matches_processor	Verified	*Token-2022 mock for {function} matches the real processor implementation, where {function} is one of `close_account`, `initialize_account`, `initialize_immutable_owner`, or `transfer_checked`.* ***Assumptions**:* • *No extensions: token accounts are `Account::BASE_LEN` and mints are `Mint::BASE_LEN`.* • *No multisig authority: the authority is not `Multisig::LEN`.* • *Token accounts and mints that the instruction reads are initialized and unpack without error.* • *Accounts that the instruction initializes are all zero.*	Report (Part I) Report (Part II)

spl_{function}_model_matches_processor		*SPL mock for {function} matches the real processor implementation, where {function} is one of* `close_account`, `initialize_account`, `initialize_immutable_owner`, *or* `transfer_checked`. ***Assumptions**:* • *No multisig authority for* `transfer`*: the authority is not* `Multisig::LEN`. • *Source and destination have different addresses on* `transfer`.	*Report*
system_{function}model_matches_processor		*System program mock for {function} matches the real processor implementation, where {function} is one of* `allocate`, `assign`, `create_account_allow_prefund`, `create_account`, `transfer`. ***Assumptions**:* • *Every account of the instruction is writable.* • *No account data was resized earlier in the transaction, in particular,* `resize_delta` *starts at 0.* • *Two different accounts have different addresses.* • *For* `create`*, the* `ZEROED_LEN` *bytes after the data of the new account are zero, and* `space` *is at most* `ZEROED_LEN` *or above* `MAX_PERMITTED_DATA_LENGTH`.	*Report*

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.